Useful facts

Open Asymptotic_Analysis-Useful-Facts.pdf

PDF Document

/assets/onedrive//Asymptotic_Analysis-Useful-Facts.pdf

when rearranging terms to proof inequality, it is helpful to group terms together and put on each site of the equation

---

Probability revision

Open prob-revision.pdf

PDF Document

/assets/onedrive//prob-revision.pdf

Average Case Analysis

Open Lec4b.pdf page 24

PDF Document

/assets/onedrive/lec/Lec4b.pdf#page=24

Outline of the proof:

1. Write expectation as average over all possible pivots:

$$A(n) = \frac{1}{n} \sum_{i=1}^{n} (A(i-1) + A(n-i)) + O(n)$$

2. Simplify the equation (manipulate and eliminate the summation) to get:

- $n \cdot A(n) = cn^2 + 2 \cdot \sum_{j=0}^{n-1} A(j)$
- $(n-1) \cdot A(n-1) = c(n-1)^2 + 2 \cdot \sum_{j=0}^{n-2} A(j)$

↓

- $n \cdot A(n) - (n-1) \cdot A(n-1) = c(2n-1) + 2A(n-1)$

3. Solve the recurrence

$$n \cdot A(n) - (n-1) \cdot A(n-1) = c(2n-1) + 2A(n-1)$$
$$n \cdot A(n) - (n+1) \cdot A(n-1) = c(2n-1)$$

- divide both sides by $n(n+1)$ and solve using telescoping method to get $A(n) = O(n \log n)$

Principle Of Deferred Decision

Open Lec5.pdf page 19

PDF Document

/assets/onedrive/lec/Lec5.pdf#page=19

If we can show that $Pr[\varepsilon | X = x] \geq p$ for all x, then $Pr[\varepsilon] \geq p$.

Union Bound

- You want to upper bound the probability that a bad event ε occurs.
- You know that $\varepsilon = \varepsilon_1 \vee \varepsilon_2 \vee \cdots \vee \varepsilon_n$.
- Union bound:
  - $Pr[\varepsilon] = Pr[\varepsilon_1] \vee [\varepsilon_2] \vee \cdots \vee [\varepsilon_n] \leq Pr[\varepsilon 1] + Pr[\varepsilon_2] + \cdots + Pr[\varepsilon_n]$.
- To make sure that $Pr[\varepsilon] \leq f$, it suffices that $Pr[\varepsilon_i] \leq \frac{f}{n}$ for each $i \in [n]$.

Markov Inequality

if X is a non-negative random variable and $a > 0$, then

$$Pr[X \geq a \cdot E[X]] \leq \frac{1}{a}$$

Open Lec5.pdf page 36

PDF Document

/assets/onedrive/lec/Lec5.pdf#page=36

Useful equations

$$1 + x \leq e^x \quad \text{for all } x \in \mathrm{R}$$

Harmonic series

$$H_n = \sum_{k=1}^{n} \frac{1}{k} = \frac{1}{1} + \frac{1}{2} + \cdots + \frac{1}{n} \approx \ln n + y$$

where $y \approx 0.577$ is Euler's constant.

Geometric series

For $r \neq 1$:

$$\sum_{k=0}^{n} r^k = \frac{r^{n+1} - 1}{r - 1} = \frac{1 - r^{n+1}}{1 - r}$$

If $|r| < 1$:

$$\sum_{k=0}^{\infty} r^k = \frac{1}{1 - r}$$

Arithmetic series

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

Summations

$$\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1}$$

$$\sum_{i=1}^{n} \log i = \log n! = \Theta(n \log n)$$

This result can be obtained using Stirling's approximation or deriving

upper bounds:

lower bounds:

$$m! = \prod_{k=1}^{m} k = \prod_{k=1}^{m} \frac{(2k-1)(2k)}{2k} = \frac{(2m-1)!! \cdot (2m)!!}{2^m \cdot m!}$$

So $(m!)^2 = \frac{(2m-1)!! \cdot (2m)!!}{2^m}$. Taking log:

$$2\log(m!) = \log((2m-1)!!) + \log((2m)!!) - m$$

Since $(2m)!! = 2^m \cdot m!$, we have $\log((2m)!!) = m + \log(m!) = \Theta(m \log m)$. The same holds for $(2m-1)!!$. Hence $\log(m!) = \Theta(m \log m)$, i.e. $\sum_{i=1}^{m} \log i = \Theta(m \log m) = \Theta(n \log n)$.

Sum of squares

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$$

Sum of cubes

$$\sum_{k=1}^{n} k^3 = \left(\frac{n(n+1)}{2}\right)^2$$

Stirling's approximation

$k! \approx \sqrt{2\pi k}\left(\frac{k}{e}\right)^k$ for large k. $lg(k!) \approx k \lg k - k \lg e + \frac{1}{2} \lg(2\pi k)$ for large k.

More precisely:

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

Binomial theorem

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}$$

Expected value

$$E[X] = \sum_{x} x \cdot Pr[X = x]$$

Combinatorics

Combination

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Permutation

$$P(n,k) = \frac{n!}{(n-k)!}$$

Strassen's Algorithm

$$\overset{C}{\begin{bmatrix} r & s \\ t & u \end{bmatrix}} = \overset{A}{\begin{bmatrix} a & b \\ c & d \end{bmatrix}} \cdot \overset{B}{\begin{bmatrix} e & f \\ g & h \end{bmatrix}}$$

- $r = P_5 + P_4 - P_2 + P_6$
- $s = P_1 + P_2$
- $t = P_3 + P_4$
- $u = P_5 + P_1 - P_3 - P_7$

- $P_1 = a \cdot (f - h)$
- $P_2 = (a + b) \cdot h$
- $P_3 = (c + d) \cdot e$
- $P_4 = d \cdot (g - e)$
- $P_5 = (a + d) \cdot (e + h)$
- $P_6 = (b - d) \cdot (g + h)$
- $P_7 = (a - c) \cdot (e + f)$

7 multiplications of $\left(\frac{n}{2} \times \frac{n}{2}\right)$ matrices: $7T\left(\frac{n}{2}\right)$ time.
18 additions of $\left(\frac{n}{2} \times \frac{n}{2}\right)$ matrices: $\Theta(n^2)$ time.
$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$
$T(n) \in \Theta\left(n^{\log_2 7}\right) = \Theta(n^{2.807\cdots})$

Karatsuba's Algorithm

1. Compute two smaller polynomial multiplications:

$$A_1(x) \times B_1(x), \quad A_2(x) \times B_2(x).$$

2. Compute one multiplication with two additions:

$$[A_1(x) + A_2(x)] \times [B_1(x) + B_2(x)].$$

3. Apply the identity, two subtractions

$$\begin{aligned} A_1(x) \times B_2(x) + A_2(x) \times B_1(x) = {} & [A_1(x) + A_2(x)] \times [B_1(x) + B_2(x)] \\ & - A_1(x) \times B_1(x) - A_2(x) \times B_2(x). \end{aligned}$$

4. Compute $C(x)$

[Generated by MarkBind 6.3.1]

Lecture 1

Open lec01a.pdf

PDF Document

/assets/onedrive/lec/lec01a.pdf

administrative info only

Lecture 1 part b

Open lec01b.pdf

PDF Document

/assets/onedrive/lec/lec01b.pdf

Introduction to Algorithms

Algorithm paradigms

- Complete Search (for example, using brute force, backtracking, branch and bound)
- Divide and Conquer (D&C)
- Dynamic Programming (DP)
- Greedy Algorithm
- Deterministic versus non-deterministic strategies
- Iterative Improvement

Analyse resource usage

- Time complexity (main focus)
- Space complexity (assume all data fits in memory)

Time-space tradeoff: sometimes we can use more space to reduce time, or vice versa.

Modelling Computation: Random Access Machine (RAM)

- count the number of basic instructions needed
- each instruction takes constant time
- Time complextiy based on input size

Example: Fibonacci numbers

iterative vs. recursive implementation (compare memory usage)

Recursive implementation

$$Fib(n) = Fib(n-1) + Fib(n-2)$$
$$Fib(n) \geq Fib(n-2) + Fib(n-2)$$
$$Fib(n) \geq 2 \cdot Fib(n-2)$$

Between 1 to n, there are $\lceil \frac{n-2}{2} \rceil$ doubling steps This takes care of odd vs even n cases
Hence $Fib(n) \geq 2^{\frac{(n-2)}{2}}$

Iterative implementation

Uses **dynamic programming**

take $O(n)$ time

Asymptotic analysis

- Measure runtime behaviour for large inputs
- ignore lower order terms and constant multiplicative factors

Big O: Upper bound

> $O(g) = \{f : \exists c > 0 \wedge n_0 > 0 s.t. \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n)\}$

$f \in O(g)$ if $\exists c > 0 \wedge n_0 \geq 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

Big Ω: Lower bound
$f \in \Omega(g)$ if $\exists c > 0$ and $n_0 > 0$ s.t. $\forall n \geq n_0$: $0 \leq c \cdot g(n) \leq f(n)$

Big Θ: Tight bound
$f \in \Theta(g)$ if $\exists c_1, c_2 > 0$ and $n_0 > 0$ s.t. $\forall n \geq n_0$: $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

$$\Theta(g) = O(g) \cap \Omega(g)$$

Little o: strict upper bound
$f \in o(g)$ if $\forall c > 0, \exists n_0 > 0$s.t. $\forall n \geq n_0$: $0 \leq f(n) < c \cdot g(n)$

basically: $o(g) = O(g) - \Theta(g)$
e.g. for any given function f, $f \in O(f)$ but $f \notin o(f)$

little ω: strict lower bound
$f \in \omega(g)$ if $\forall c > 0, \exists n_0 > 0$s.t. $\forall n \geq n_0$: $0 \leq c \cdot g(n) < f(n)$

$\omega(g) = \Omega(g) - \Theta(g)$

Determining complexity by taking limits

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \quad \to f \in o(g)$$
$$\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty \quad \to f \in O(g)$$
$$0 < \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty \quad \to f \in \Theta(g)$$
$$\lim_{n\to\infty} \frac{f(n)}{g(n)} > 0 \quad \to f \in \Omega(g)$$
$$\lim_{n\to\infty} \frac{f(n)}{g(n} = \infty \quad \to f \in \omega(g)$$

Proofs

Proof:

By definition of limit, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$, means
$\forall \epsilon > 0, \exists n_0 > 0$, such that $\forall n \geq n_0$,
$\frac{f(n)}{g(n)} < \epsilon$

Hence, for any constant $c > 0$ (i.e., we can set $\epsilon = c$), $\exists n_0 > 0$, such that $\forall n \geq n_0$,
$f(n) < \epsilon \cdot g(n)$, i.e.,
$f(n) < c \cdot g(n)$,
$f \in o(g)$

We will prove at least one other during Tut01

Proofs from Tutorial 1

Solution

Open tutorial01-answers.pdf

PDF Document

/assets/onedrive/tut/tutorial01-answers.pdf

[Generated by MarkBind 6.3.1]

Tutorial 1

Open tutorial01-questions.pdf

PDF Document

/assets/onedrive/tut/tutorial01-questions.pdf

Slides

Open tutorial01-slides.pdf

PDF Document

/assets/onedrive/tut/tutorial01-slides.pdf

Solution

Open tutorial01-answers.pdf

PDF Document

/assets/onedrive/tut/tutorial01-answers.pdf

Lecture 2 part a

Open Lec2.pdf

PDF Document

/assets/onedrive/lec/Lec2.pdf

Solving a recurence

Telescoping

- rearrange the pattern, expand the summation and cancel out terms

Substitution

1. Guess a solution (using master's theorem etc)
2. Verify by induction

> Can introduce extra terms to 'kill' annoying terms when proving big O

Open Lec2.pdf page 23

PDF Document

/assets/onedrive/lec/Lec2.pdf#page=23

Recursion Tree

Open Lec2.pdf page 30

PDF Document

/assets/onedrive/lec/Lec2.pdf#page=30

Master Theorem

for recurrences of the form:

$$T(n) = a \cdot T(\frac{n}{b}) + f(n)$$
$$a, b \geq 1, f \in \Omega(1)$$

Open Lec2.pdf page 41

PDF Document

/assets/onedrive/lec/Lec2.pdf#page=41

case 1: dominated by the leaves **case 2**: dominated by the top level f(n)

read the lecture note for proofs

Substituting n to solve recurrences

Open Lec2.pdf page 47

PDF Document

/assets/onedrive/lec/Lec2.pdf#page=47

Deriving the tight bound

Substitution: Let $n = 2^k$, so $k = \log_2 n$, and define $S(k) = T(2^k)$.

The recurrence becomes:

$$S(k) = S(k-1) + 2^{\sqrt{k}}$$

Unrolling:

$$S(k) = S(0) + \sum_{i=1}^{k} 2^{\sqrt{i}}$$

So the key is evaluating $\sum_{i=1}^{k} 2^{\sqrt{i}}$.

Estimating the sum

Integral approximation: Substitute $u = \sqrt{x}$, $dx = 2u\,du$:

$$\int_1^k 2^{\sqrt{x}}\,dx = 2\int_1^{\sqrt{k}} u \cdot 2^u\,du$$

Integration by parts gives:

$$\int u \cdot 2^u\,du = \frac{u \cdot 2^u}{\ln 2} - \frac{2^u}{(\ln 2)^2}$$

Evaluating at $\sqrt{k}$, the dominant term is $\frac{\sqrt{k} \cdot 2^{\sqrt{k}}}{\ln 2}$, so the integral is $\Theta(\sqrt{k} \cdot 2^{\sqrt{k}})$.

Why the last term alone isn't enough: The ratio of consecutive terms $2^{\sqrt{i}}/2^{\sqrt{i-1}} = 2^{\sqrt{i}-\sqrt{i-1}} \approx 2^{1/(2\sqrt{i})} \to 1$, so the terms don't decay geometrically — many terms near $i = k$ contribute significantly. Specifically, for $i = k - j$:

$$2^{\sqrt{k-j}} \approx 2^{\sqrt{k}} \cdot 2^{-j/(2\sqrt{k})}$$

Summing the geometric-like tail gives a factor of $\Theta(\sqrt{k})$ multiplying the largest term.

Result

Substituting back $k = \log_2 n$:

$$\boxed{T(n) = \Theta(\sqrt{\log n} \cdot 2^{\sqrt{\log n}})}$$

[Generated by MarkBind 6.3.1]

Tutorial 2

Open tutorial02-questions.pdf

PDF Document

/assets/onedrive/tut/tutorial02-questions.pdf

Slides

Open tutorial02-slides.pdf

PDF Document

/assets/onedrive/tut/tutorial02-slides.pdf

Solution

Open tutorial02-answers.pdf

PDF Document

/assets/onedrive/tut/tutorial02-answers.pdf

Proof of Correctness

Iterative version

Open Lec3a.pdf page 7

PDF Document

/assets/onedrive/lec/Lec3a.pdf#page=7

important to also consider elements not being acted on in a step (e.g. state that it stays the same)

- **Initialization**: $n = 1$, trivially/vacuously true?
- **Maintenance**: if $n - 1$ holds, n is true
- **Termination**: imagine there's a $k + 1^{th}$ step, nothing changes

Open Lec3a.pdf page 17

PDF Document

/assets/onedrive/lec/Lec3a.pdf#page=17

Recursive version

- Base case
- Inductive step

Open Lec3a.pdf page 43

PDF Document

/assets/onedrive/lec/Lec3a.pdf#page=43

[Generated by MarkBind 6.3.1]

Divide and Conquer

Open Lec3b.pdf

PDF Document

/assets/onedrive/lec/Lec3b.pdf

e.g. Merge sort

1. Divide the problem into smaller subproblems.
2. consider splitting and combining cost
3. Solve the subproblems recursively.
4. Combine the subproblem solutions to get the solution of the full problem.

Reformulate to break problem into subproblems

Allow divide and conquer method to be applied to a wider range of problems.
e.g. using clever ways to reduce the number of subproblems, or to reduce the cost of splitting and combining.

Reducing number of subproblems

Open Lec3b.pdf page 27

PDF Document

/assets/onedrive/lec/Lec3b.pdf#page=27

Tutorial 3

Open tutorial03-questions.pdf

PDF Document

/assets/onedrive/tut/tutorial03-questions.pdf

Slides

Open tutorial03-slides.pdf

PDF Document

/assets/onedrive/tut/tutorial03-slides.pdf

Solution

Open tutorial03-answers.pdf

PDF Document

/assets/onedrive/tut/tutorial03-answers.pdf

Lecture 4 part a

Open Lec4a.pdf

PDF Document

/assets/onedrive/lec/Lec4a.pdf

Comparison based algorithms

- Comparison-based algorithms:
  - Elements can only be compared with each other: <, ≤, =, >, ≥
  - No other information of the elements can be used.

Open Lec4a.pdf page 16

PDF Document

/assets/onedrive/lec/Lec4a.pdf#page=16

Stirling's approximation: For large n, n! can be approximated as:

$$\log(n!) \approx n \log n - n \log e + O(\log n) \subseteq \Theta(n \log n)$$

Proving the lower bound

In general, we can show that there is at least l leaves in the decision tree, where $l \geq n$ is the number of possible outcomes. Since in each comparison, we can only split the possibilities into c branches, the height of the tree must be at least $\log_c l$. Therefore, the number of comparisons must be at least $\log_c n$.

In the case of sorting, we have $n!$ possible outcomes (permutations), so the number of comparisons must be at least $\log_c n! \approx n \log n$.

Breaking the barrier: Non-comparison sorts

Counting sort, radix sort, bucket sort etc.

> radix sort: sort lower significant digits first, then more significant digits.
> bounded by the number of digits (in base n), not the number of elements

See assignement for more details

[Generated by MarkBind 6.3.1]

Average case analysis

Open Lec4b.pdf

PDF Document

/assets/onedrive/lec/Lec4b.pdf

Quicksort

Worst case analysis

Assuming that the pivot is always the smallest element, we have:
$T(n) = T(n-1) + O(n) = O(n^2)$

Average case analysis

Open Lec4b.pdf page 24

PDF Document

/assets/onedrive/lec/Lec4b.pdf#page=24

Outline of the proof:

1. Write expectation as average over all possible pivots:

$$A(n) = \frac{1}{n}\sum_{i=1}^{n}(A(i-1) + A(n-i)) + O(n)$$

2. Simplify the equation (manipulate and eliminate the summation) to get:

- $n \cdot A(n) = cn^2 + 2 \cdot \sum_{j=0}^{n-1} A(j)$
- $(n-1) \cdot A(n-1) = c(n-1)^2 + 2 \cdot \sum_{j=0}^{n-2} A(j)$

↓

- $n \cdot A(n) - (n-1) \cdot A(n-1) = c(2n-1) + 2A(n-1)$

3. Solve the recurrence

$$n \cdot A(n) - (n - 1) \cdot A(n - 1) = c(2n - 1) + 2A(n - 1)$$
$$n \cdot A(n) - (n + 1) \cdot A(n - 1) = c(2n - 1)$$

- divide both sides by $n(n + 1)$ and solve using telescoping method to get $A(n) = O(n \log n)$

Desirable characteristics of a sorting algorithm

- Fast (avere case and worst case)
- Stable
  - If $A[i] = A[j] \wedge i < j$, then $A[i]$ comes before $A[j]$ in the output
  - insertion sort, proper implementation of merge sort are stable, quicksort is not
- Memory efficient (in-place)

[Generated by MarkBind 6.3.1]

Tutorial 4

Open tutorial04-questions.pdf

PDF Document

/assets/onedrive/tut/tutorial04-questions.pdf

Slides

Open tutorial04-slides.pdf

PDF Document

/assets/onedrive/tut/tutorial04-slides.pdf

Solution

Open tutorial04-answers.pdf

PDF Document

/assets/onedrive/tut/tutorial04-answers.pdf

Randomized algorithm

Open Lec5.pdf

PDF Document

/assets/onedrive/lec/Lec5.pdf

- Monte Carlo algorithm (random output)
- Las Vegas algotirhm (random runtime)
  - can be turned into Monte Carlo via Markov's inequality

Freivald's algorithm

Open Lec5.pdf page 17

PDF Document

/assets/onedrive/lec/Lec5.pdf#page=17

Monte Carlo algorithm for verifying matrix multiplication.
Relies on the fact that if we defer random choice of $u_i \in \{0, 1\}$, we can show that the probability of error is at most $\frac{1}{2}$.

Principle of deferred decision

Open Lec5.pdf page 19

PDF Document

/assets/onedrive/lec/Lec5.pdf#page=19

If we can show that $Pr[\varepsilon | X = x] \geq p$ for all x, then $Pr[\varepsilon] \geq p$.

Union bound

- You want to upper bound the probability that a bad event ε occurs.
- You know that $\varepsilon = \varepsilon_1 \vee \varepsilon_2 \vee \cdots \vee \varepsilon_n$.
- Union bound:
  - $Pr[\varepsilon] = Pr[\varepsilon_1] \vee [\varepsilon_2] \vee \cdots \vee [\varepsilon_n] \leq Pr[\varepsilon 1] + Pr[\varepsilon_2] + \cdots + Pr[\varepsilon_n]$.
- To make sure that $Pr[\varepsilon] \leq f$, it suffices that $Pr[\varepsilon_i] \leq \frac{f}{n}$ for each $i \in [n]$.

Markov's inequality

if X is a non-negative random variable and $a > 0$, then

$$Pr[X \geq a \cdot E[X]] \leq \frac{1}{a}$$

Open Lec5.pdf page 36

PDF Document

/assets/onedrive/lec/Lec5.pdf#page=36

Randomized quicksort

Open Lec5.pdf page 57

PDF Document

/assets/onedrive/lec/Lec5.pdf#page=57

here, between means comparing the two elements, not everything in between

[Generated by MarkBind 6.3.1]

Tutorial 5

Open tutorial05-questions.pdf

PDF Document

/assets/onedrive/tut/tutorial05-questions.pdf

Slides

Open tutorial05-slides.pdf

PDF Document

/assets/onedrive/tut/tutorial05-slides.pdf

Solution

Open tutorial05-answers.pdf

PDF Document

/assets/onedrive/tut/tutorial05-answers.pdf

Greedy algorithms

Open Lec7.pdf page 2

PDF Document

/assets/onedrive/lec/Lec7.pdf#page=2

1. Cast the problem where we have to make a choice and are left with one subproblem to solve.
2. Prove that there is always an optimal solution to the original problem that makes the greedy choice, so the greedy choice is safe.
3. Use optimal substructure to show that we can combine an optimal solution to the subproblem with the greedy choice to get an optimal solution to the original problem.

Dynamic vs Greedy

- Dynamic: avoid overlapping work in optimal substructure
- Greedy: go down on the optimal path on each step i.e. no 'useless work'

Greedy choice property

A locally optimal choice is globally optimal.